

Методичка: централизованный мониторинг Linux-серверов в Zabbix

Темы: Zabbix Server, Agent, Host, Template, Item, Trigger, Severity, dashboards, active/passive checks

ОС сервера: Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

ОС клиента: Debian/Ubuntu или Windows 10/11, если используется

Среда: VirtualBox

Формат: самостоятельная практическая работа

Ориентировочное время: 4-6 академических часов

1. Что настроим

В этой работе разворачиваем или проверяем серверный сервис Linux-инфраструктуры. Команды выполняем на Debian/Ubuntu, а результат подтверждаем локально и с другого узла.

Используем узлы: zbx01, web01, vpn01, backup01.

Главная идея работы:

Мониторинг считается настроенным не тогда, когда хост добавлен в интерфейс, а тогда, когда метрики приходят, trigger срабатывает при отказе и исчезает после восстановления.

2. Схема учебного стенда

Узел	Роль	ОС	Сетевые адаптеры	IP-адрес
srv01	основной сервер сервиса	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.10/24
srv02	второй сервер или клиент	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.20/24
client01	клиентская проверка	Debian/Ubuntu или Windows	внутренняя сеть	192.168.10.30/24

VirtualBox NAT для пакетов

```
|
srv01 ---- linux-lab 192.168.10.0/24 ---- srv02/client01
```

Если используем имена, добавляем DNS или `/etc/hosts` и проверяем `getent hosts`.

3. Необходимые ресурсы и предварительные условия

Нужны:

- готовая VM Zabbix Server или сервер для установки Zabbix;
- Linux-узлы `web01`, `vpn01`, `backup01`;
- доступ к web-интерфейсу Zabbix;

- пакеты `zabbix-agent2` на клиентах.

4. Подготовка виртуальных машин

```
hostnamectl  
ip -br address  
ip route  
resolvectl status  
systemctl --failed
```

Проверяем имена:

```
getent hosts srv01.company.test  
getent hosts srv02.company.test
```

Если имена не находятся, добавляем записи в `/etc/hosts`.

Самопроверка этапа

- NAT работает для установки пакетов;
- внутренняя сеть доступна;
- имена узлов разрешаются;
- снимки ВМ созданы.

Если результат отличается от ожидаемого, дальше пока не продолжаем. Сначала проверяем этот этап.

5. Адресный план или план ресурсов

Ресурс	Значение
Основной сервер	<code>srv01</code> , <code>192.168.10.10</code>

Второй сервер	srv02, 192.168.10.20
Клиент	client01, 192.168.10.30
Учебный домен	company.test
Данные сценария	по теме лекции

План работы:

- подготовить Zabbix Server или готовую VM;
- установить agent на Linux-хосты;
- добавить hosts и host groups;
- привязать Linux templates;
- проверить CPU, RAM, disk, network и service checks;
- создать trigger по диску или службе;
- смоделировать отказ и восстановление.

6. Исходная проверка

```
cat /etc/os-release
ip -br address
ip route
ss -lntup
systemctl --failed
df -h
```

Самопроверка этапа

- ОС и сеть проверены;

- нужные порты свободны или понятно, какая служба их использует;
- свободного места достаточно;
- failed-службы разобраны до настройки.

7. Пошаговая настройка

7.1. Подготовка пакетов

```
sudo apt update
```

Если пакеты не скачиваются, проверяем NAT, DNS и маршрут.

7.2. Основная настройка

```
sudo apt install zabbix-agent2 -y
sudo cp /etc/zabbix/zabbix_agent2.conf /etc/zabbix/zabbix_agent2.conf.bak
sudo nano /etc/zabbix/zabbix_agent2.conf
```

Ключевые строки:

```
Server=192.168.10.10
ServerActive=192.168.10.10
Hostname=web01
```

Проверяем и запускаем:

```
sudo systemctl enable --now zabbix-agent2
systemctl status zabbix-agent2 --no-pager
ss -lntup | grep 10050
```

В web-интерфейсе Zabbix создаём host, группу `Linux servers`,
привязываем template `Linux by Zabbix agent`.

Самопроверка этапа

```
systemctl status zabbix-agent2 --no-pager
sudo journalctl -u zabbix-agent2 -n 50 --no-pager
ss -lntup | grep 10050
```

В Zabbix проверяем Latest data, Problems и график CPU/disk. Для отказа останавливаем тестовую службу и ждём problem event.

Если результат отличается от ожидаемого, дальше пока не продолжаем.

Сначала проверяем этот этап.

7.3. Восстановление или откат

Для backup, мониторинга, Docker и HAProxy обязательно проверяем восстановление или повторное создание результата: восстановить файл, вернуть сервис, пересоздать контейнер с сохранением данных или вернуть backend в балансировку.

8. Проверка итогового результата

```
systemctl status zabbix-agent2 --no-pager  
sudo journalctl -u zabbix-agent2 -n 50 --no-pager  
ss -lntup | grep 10050
```

В Zabbix проверяем Latest data, Problems и график CPU/disk. Для отказа останавливаем тестовую службу и ждём problem event.

Границы результата:

- учебный стенд не заменяет промышленную отказоустойчивую архитектуру;
- тестовые пароли и ключи нельзя переносить в реальную среду;
- сервис считается готовым только после клиентской проверки, журнала и проверки восстановления, если она относится к теме.

9. Диагностика типовых ошибок

Симптом	Вероятная причина	Проверка	Исправление
Данные не приходят	Неверный Server/Hostname или firewall	<code>journalctl -u zabbix-agent2,</code> Latest data	Исправить config и firewall
Trigger не срабатывает	Нет item или порог неверный	Zabbix Latest data, trigger expression	Проверить item и expression
Ложные срабатывания	Порог слишком жёсткий	История метрики	Подобрать порог по baseline
Пакет не устанавливается	Нет доступа к репозиториям	<code>ip route, getent hosts deb.debian.org</code>	Проверить NAT и DNS
Сервис не отвечает	Порт не слушается или firewall блокирует	<code>ss -lntup, nc -vz</code>	Проверить службу и firewall

Места на диске мало	Логи, backup или volumes растут	<code>df -h, du -xhd1</code>	Очистить или расширить хранилище по плану
------------------------	------------------------------------	------------------------------	--

10. Итоговая самостоятельная проверка

- ☐ Стенд соответствует схеме.
- ☐ Имена и адреса согласованы.
- ☐ Пакеты установлены.
- ☐ Сервис или сценарий запущен.
- ☐ Проверочные команды подтверждают результат.
- ☐ Клиентская проверка проходит.
- ☐ Восстановление или повторное создание результата проверено.
- ☐ В журналах нет необъяснённых ошибок.
- ☐ Одна типовая ошибка найдена и исправлена.

11. Что приложить к отчёту

К отчёту прикладываем схему, адресный план, ключевой фрагмент конфигурации, вывод проверки, клиентский результат, журнал/метрики, описание исправленной ошибки и результат восстановления.

12. Контрольные вопросы

1. Какую задачу решает технология этой лекции?
2. Какие ресурсы нужно оценить до внедрения?
3. Какая команда подтверждает основной результат?
4. Как проверить сервис с клиента?
5. Что проверить в журналах?
6. Чем учебная настройка отличается от промышленной?
7. Как выполнить безопасный откат?
8. Что приложить к отчёту?